

Information Retrieval - News Recommender System

Pietro Terribile - IN2300014, A.Y. 2025 - 2026

Index

- [Introduction](#)
- [Dataset preview](#)
- [Implicit-feedback collaborative filtering](#)
- [Implementation choices](#)
- [Evaluation Methodology and Metrics](#)
- [Results](#)
- [Execution pipeline](#)
- [Conclusions](#)
- [References](#)

Introduction

This project consists in a recommender system for news articles trained on the Microsoft News Dataset (actually, MIND-small, a look-alike of the wider, original and no more available dataset: <https://huggingface.co/datasets/huyva/MIND-small>).

The system is based on Hu et al.'s *Collaborative Filtering for Implicit Feedback Datasets* formulation and it is adapted to exploit MIND's features in the data given: the dataset records not only clicked articles, but also articles that were shown and not clicked.

The project therefore distinguishes among three types of user - article relationships:

- positive interactions, such as articles in a user's history or clicked impressions
- observed negative interactions, corresponding to displayed but non-clicked articles
- unobserved interactions, for which the system has no direct evidence

This distinction, merged with the implicit-feedback formulation, allows to treat differently unseen items and uninteracted ones.

Dataset preview

The MIND data is session-oriented and comes already in 'train_split' and 'dev_split', both containing 'behaviors.tsv' and 'news.tsv'. The training repository also contains 'popularity_data.csv': in each row, in a decreasing order, the most popular articles and the clicks that have generated.

Each row of 'behaviors.tsv' describes a user session containing:

- a user identifier;
- a timestamp;
- a history of previously clicked articles;
- a list of impressions with binary click labels ($\mathbb{N}^{*****}-0$ if the article has been suggested to the user but he did not click on it, $\mathbb{N}^{*****}-1$ if he clicked on it).

Each row of 'news.tsv' contains:

- the article ID
- the category
- the subcategory
- the title and abstract
- other informations and metadata that are not exploited by the system

This format is naturally suited to session-based ranking: given a set of displayed articles, predict which ones the user will click.

WALS, however, is formulated over a global user - item preference matrix. That's why the project lies on the *Collaborative Filtering for Implicit Feedback Datasets* formulation.

Implicit-feedback collaborative filtering

Let:

$$C_{pos} \in \mathbb{R}^{n_u \times n_i}$$

be the positive-feedback matrix, the sparse matrix containing the positive interactions of all users (both the news in user's history and those flagged with -1) and:

$$M_{neg} \in \mathbb{R}^{n_u \times n_i}$$

be the matrix of observed non-clicks (the suggested and ignored items, flagged with -0 , are the 1 in this sparse matrix).

The model learns user and item factors:

$$U \in \mathbb{R}^{n_u \times f}, \quad V \in \mathbb{R}^{n_i \times f}$$

and estimates the compatibility between user u and article i using:

$$\hat{r}_{ui} = x_u^T v_i.$$

where x_u and v_i are respectively the user-factor vector and the item-factor vector, both $\in \mathbb{R}^f$.

Following the implicit-feedback framework proposed by Hu et al., raw user feedback is decomposed into two distinct variables: a binary preference indicator p_{ui} and a confidence measure c_{ui} .

The preference p_{ui} indicates whether an interaction occurred:

$$p_{ui} = \begin{cases} 1 & \text{if } (u, i) \in C_{\text{pos}} \quad (\text{history or clicked impression}) \\ 0 & \text{otherwise} \quad (\text{unobserved or observed non-click}) \end{cases}$$

To handle the three interaction levels available in MIND without treating non-clicks as severe negative targets ($p_{ui} = -1$), the confidence c_{ui} is scaled as follows :

- **Positive interactions** $(u, i) \in C_{\text{pos}}$: $c_{ui} = 1 + \alpha$ (with $\alpha = 40.0$)
- **Observed non-clicks** $(u, i) \in M_{\text{neg}}$: $c_{ui} = 1 + \alpha_{\text{neg}}$ (with $\alpha_{\text{neg}} = 20.0$)
- **Unobserved entries**: $c_{ui} = 1.0$ (baseline confidence)

The parameters $U = [x_1, \dots, x_{n_u}]^\top$ and $V = [v_1, \dots, v_{n_i}]^\top$ are estimated by minimizing the weighted, L_2 -regularized least-squares loss:

$$\min_{U, V} \sum_{u=1}^{n_u} \sum_{i=1}^{n_i} c_{ui} (p_{ui} - x_u^\top v_i)^2 + \lambda \left(\sum_{u=1}^{n_u} \|x_u\|_2^2 + \sum_{i=1}^{n_i} \|v_i\|_2^2 \right)$$

where $\lambda = 0.01$ and the latent dimension is $f = 50$.

User Factors Update (fixing V)

The paper (Hu et al., eq. 4) derives the closed-form update for x_u by differentiating the weighted loss with respect to x_u and setting the gradient to zero:

$$x_u = (V^\top C^u V + \lambda I)^{-1} V^\top C^u p(u)$$

where C^u is the diagonal $n_i \times n_i$ matrix with $C_{ii}^u = c_{ui}$, and $p(u) \in \mathbb{R}^{n_i}$ is the preference vector for user u .

The paper then exploits the algebraic identity:

$$V^\top C^u V = V^\top V + V^\top (C^u - I) V$$

Since $(C^u - I)$ is non-zero only for the few items observed by user u (far fewer than the total catalog size n_i), this algebraic trick avoids iterating over all unobserved news and reduces the per-user update cost from $O(f^2 n_i)$ to $O(f^2 |\text{observed}| + f^3)$, making the overall training algorithm linear in the total number of observations $\mathcal{N}$.

Applying the same algebraic decomposition, $(C^u - I)$ now has non-zero entries equal to α on positive interactions and α_{neg} on observed non-clicks. Expanding $V^\top C^u V$ accordingly and substituting into the normal equations gives:

$$A_u x_u = b_u$$

where:

$$A_u = \underbrace{V^\top V}_{\text{global, precomputed}} + \underbrace{\alpha \sum_{i \in \text{pos}(u)} v_i v_i^\top}_{\text{positive correction}} + \underbrace{\alpha_{\text{neg}} \sum_{i \in \text{neg}(u)} v_i v_i^\top}_{\text{negative correction}} + \lambda I_f$$

and:

$$b_u = V^\top C^u p(u) = \sum_{i \in \text{pos}(u)} (1 + \alpha) v_i$$

The right-hand side receives contributions only from positive interactions because $p_{ui} = 0$ for all non-clicks, making $c_{ui} \cdot p_{ui} \cdot v_i = 0$ regardless of c_{ui} . Observed non-clicks contribute exclusively to A_u through $\alpha_{\text{neg}} v_i v_i^\top$: this increases the steepness of the cost function along the direction of rejected items, moving away x_u from those directions without assigning an explicit negative preference target.

Item Factors Update (fixing U)

Symmetrically, the paper (eq. 5) gives:

$$v_i = (U^\top C^i U + \lambda I_f)^{-1} U^\top C^i p(i)$$

where C^i is diagonal over users and $p(i) \in \mathbb{R}^{n_u}$ contains the preferences for item i . Applying the same three-level extension leads to:

$$A_i v_i = b_i$$

with:

$$A_i = \underbrace{U^\top U}_{\text{global, precomputed}} + \underbrace{\alpha \sum_{u \in \text{pos}(i)} x_u x_u^\top}_{\text{positive correction}} + \underbrace{\alpha_{\text{neg}} \sum_{u \in \text{neg}(i)} x_u x_u^\top}_{\text{negative correction}} + \lambda I_f$$

$$b_i = \sum_{u \in \text{pos}(i)} (1 + \alpha) x_u$$

Therefore, since $(C^i - I)$ is non-zero only for the few users who observed item i (far fewer than the total user base n_u), updating each item vector scales with the number of interactions for that specific news rather than the entire user population. Precomputing $U^\top U$ in $O(f^2 n_u)$ once per sweep yields a total item-step complexity of $O(f^2 \mathcal{N} + f^3 n_i)$, again linear in the data size.

Implementation choices

Unified Joint Indexing

By inspecting the data, many of the articles in the dev split were not present in the training data: since for a recommender it is assumed that the system has access to the whole catalog, I decided to merge the two indices. Without leaking test interactions, `data_loader.py` builds an overarching index over `train` $\cup$ `dev` :

- The mapping covers all unique users and news IDs across both splits.
- `train_data.C_pos` and `train_data.M_neg` only contain interaction signals from `data/train/behaviors.tsv` .
- Novel dev-only items occupy zeroed columns during training and converge to near-zero representations under regularization, avoiding label leakage while remaining indexable during inference.

Cold-Start Handling: Online Fold-In

For unseen users or users without training interactions (e.g., dev-only users), the model avoids re-running global matrix factorization by employing an online **fold-in** step:

$$x_u = \left(V^\top V + \alpha \sum_{i \in \text{history}} v_i v_i^\top + \alpha_{\text{neg}} \sum_{j \in \text{neg_pool}} v_j v_j^\top + \lambda I_f \right)^{-1} \left((1 + \alpha) \sum_{i \in \text{history}} v_i \right)$$

If an incoming user has no recorded history at all, the system gracefully falls back to a global popularity rank derived from `popularity_data.csv` .

Two-Stage Scoring and Hybrid Re-ranking

The recommendation service in `recommender.py` is designed with a multi-stage architecture :

1. Candidate Retrieval:

- *Global Demo Mode*: Computes Vx_u across all items in one matrix-vector product, removes training/session history, and selects the top $N = 500$ candidates via `argpartition` .
- *Evaluation Mode*: Directly restricts candidate evaluation to the specific impression pool (10 - 50 items) associated with the session.

2. Min-Max Score Normalization:

$$\hat{s}_{\text{wals}}(i) = \frac{x_u^\top v_i - \min_{j \in \mathcal{C}}(x_u^\top v_j)}{\max_{j \in \mathcal{C}}(x_u^\top v_j) - \min_{j \in \mathcal{C}}(x_u^\top v_j)}$$

3. Popularity Re-ranking:

$$\text{score}(u, i) = (1 - w_{\text{pop}}) \cdot \hat{s}_{\text{wals}}(i) + w_{\text{pop}} \cdot s_{\text{pop}}(i)$$

with default weight $w_{\text{pop}} = 0.05$.

Evaluation Methodology and Metrics

Because news recommendation benchmarks such as MIND evaluate whether a model can rank the clicked item at the top of a specific impression list, offline validation in `main.py` is conducted over the session's **impression pool** rather than over the entire 65k catalog.

The evaluation suite implemented in `metrics.py` includes :

1. Precision@k and Recall@k:

$$\text{Precision@}k = \frac{|\text{Top-}k \cap \text{Rel}|}{k}, \quad \text{Recall@}k = \frac{|\text{Top-}k \cap \text{Rel}|}{|\text{Rel}|}$$

- **Precision@k** measures the "purity" of the top recommendations: out of the top k positions displayed on the screen, how many are actual relevant clicks?
- **Recall@k** measures the "coverage" of user intent: what proportion of all positive interactions in that session was successfully captured within the top k slots?

2. Mean Average Precision (MAP):

$$\text{AP} = \frac{1}{|\text{Rel}|} \sum_{k=1}^{|\mathcal{C}|} \text{Precision@}k \cdot \mathbb{I}(\text{item}_k \in \text{Rel}), \quad \text{MAP} = \frac{1}{|U_{\text{eval}}|} \sum_{u \in U_{\text{eval}}} \text{AP}_u$$

- **MAP** evaluates the ranking quality across all positive items in a session, penalizing relevant articles that appear lower in the list, giving a global rank-aware measure for the entire session pool. If a session contains two clicked articles, placing them at positions (1, 2) yields $\text{AP} = 1.0$, whereas placing them at positions (1, 10) yields $\text{AP} = \frac{1+2/10}{2} = 0.60$.

3. Normalized Discounted Cumulative Gain (NDCG@k):

$$\text{DCG@}k = \sum_{r=1}^k \frac{\mathbb{I}(\text{item}_r \in \text{Rel})}{\log_2(r+1)}, \quad \text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}$$

where $\text{IDCG@}k$ is the ideal discounted gain obtained by placing all $\min(|\text{Rel}|, k)$ relevant articles at the top ranks .

$\text{NDCG@}k$ is a widely used ranking metric in modern information retrieval and competitive news recommendation benchmarks (including the official MIND challenge).

While MAP considers relative hit positions, $\text{NDCG@}k$ applies a **logarithmic position discount** based on absolute rank:

- A click at rank 1 contributes $\frac{1}{\log_2(2)} = 1.000$
- A click at rank 2 contributes $\frac{1}{\log_2(3)} \approx 0.631$
- A click at rank 5 contributes $\frac{1}{\log_2(6)} \approx 0.387$

This metric was employed because it reflects real-world user behavior on news feeds, where user attention decays sharply below the top 3 - 5 headlines.

Results

Quantitative Evaluation

Offline evaluation was conducted on all the 73.152 valid development sessions in `dev/behaviors.tsv` (sessions containing at least one clicked article). The model re-ranks the impression pool of each session (10 - 50 candidates), and metrics are computed against the clicked articles as ground truth.

The performance of the proposed WALS model is benchmarked against two standard baselines:

- **Random Baseline:** uniform random shuffling of the impression pool.
- **Popularity Baseline:** sorting the impression pool in decreasing order of global training click count (`popularity_data.csv`).

Metric	Random Baseline	Popularity Baseline	Proposed WALS
NDCG@5	0.2220	0.2373	0.2637
NDCG@10	0.2853	0.3010	0.3243
MAP	0.2298	0.2438	0.2649
Precision@5	0.0996	0.1037	0.1133
Precision@10	0.1001	0.1024	0.1064
Recall@10	0.5216	0.5379	0.5596

Discussion

The empirical evaluation reveals a clear and consistent hierarchy across all ranking-sensitive metrics:

$$\text{WALS} > \text{Popularity Baseline} > \text{Random Baseline}$$

Four key analytical findings emerge from the results:

Superiority on rank-aware metrics: The proposed WALS model achieves $\text{MAP} = 0.2649$ and $\text{NDCG@5} = 0.2637$, outperforming the Popularity baseline by $+8.7\%$ and $+11.1\%$ and the Random baseline by $+15.3\%$ and $+18.8\%$, respectively. This confirms that collaborative latent factor modeling, paired with three-tier confidence weighting, extracts personalized user intent effectively.

Top concentration (Precision@5 vs. NDCG@5): While global popularity represents a strong baseline in news domains due to herd reading behavior, WALS demonstrates its primary advantage in the top visual ranks. Precision@5 increases from 0.1037 (Popularity) to **0.1133** (WALS, +9.3%), directly driving the increase in NDCG@5. The model succeeds in pushing clicked articles to the very top of the recommendation list, which is key in production feeds where user attention decays sharply after the first 3 to 5 items.

High intent capture (Recall@10 = 0.5596): Within the top 10 recommended slots of the session pool, WALS recovers **55.96%** of all positive clicks, surpassing both the Popularity baseline (53.79%) and Random ordering (52.16%).

Structural Saturation of Precision@10: remains more or less constant across all three methods. This is a characteristic of the dataset rather than a limitation of the model: each MIND session contains an average of ~ 2 relevant clicks within a pool of ~ 20 displayed impressions. When evaluating at $k = 10$, any algorithm retrieving half the pool has an expected precision naturally bounded around $2/20 = 0.10$. Consequently, Precision@10 is non-discriminative, making rank-sensitive metrics (MAP and NDCG) the true benchmark indicators for news recommendation quality.

Qualitative Evaluation: User Profiles

The model is tested on a representative user, **U1** (dev-only, fold-in on 12 history articles and 1 observed non-click). The top-10 recommendations span finance, sports, and news — categories consistent with a history dominated by political and sports content. Three of the ten nearest neighbors in the latent space correspond to articles actually present in the user's history.

Execution pipeline

From terminal:

1. `pip install -r requirements.txt`
2. `python data_loader.py`
 - └ Downloads raw dataset and constructs joint index & feedback matrices (C_{pos} , M_{neg})

|
▼
3. `python train.py` *<-- uses wals.py*
 - └ Trains WALS embeddings (U , V) on training interactions and serializes all artifacts/

|
▼
4. `python main.py / demo_web.py` *<-- use metrics.py & recommender.py (Loads artifacts, handles c*
 - └ Executes offline benchmark evaluation on dev impressions and serves the interactive UI

Conclusions

This project implements a complete WALS-based news recommendation pipeline, from raw MIND logs to interactive recommendations and session-level evaluation.

Its central contribution is the extension of implicit-feedback WALS to distinguish positive interactions, displayed non-clicks, and unobserved pairs. Positive interactions strongly orient the latent representations, while observed non-clicks increase the confidence of the zero-preference constraint without being interpreted as explicit dislike.

References

- [1] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative Filtering for Implicit Feedback Datasets," in *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 263–272. DOI: 10.1109/ICDM.2008.22.
- [2] AI Assistance Notice: Parts of the codebase and debugging workflows were developed with the support of Claude (Anthropic). The theoretical modeling, experimental design, and final analysis remain the sole responsibility of the author.